

Unit I – Chapter 2

- Introduction
- Characteristics of Database approach
- Actors on the Scene
- Workers behind the scene
- Advantages of using DBMS approach
- Data models, schemas and instances
- Three-schema architecture and data independence
- Database languages and interfaces
- The database system environment
- Centralized and client-server architectures
- Classification of Database Management systems,
- Entity-Relationship Model:
- Using high level Conceptual data models for Database Design
- A Sample Database Application
- Entity types, Entity sets Attributes and Keys
- Relationship types, Relationship Sets,
- Roles and Structural Constraints
- Weak Entity Types.

DATA MODELS, SCHEMAS, AND INSTANCES (1)

-
- The architecture of DBMS
 - **Monolithic systems** - the whole DBMS software package is **one tightly integrated system**
 - Modern DBMS packages that are **modular in design**, with a **client/server system architecture**.
 - **Client module** - runs on user workstation or personal computer. Application programs and user interfaces that access the database run in the client module. User-friendly interfaces such as **forms- or menu based GUIs** (Graphical User Interfaces).
 - **Server module** - handles **data storage, access, search**, and other functions. Mainframe computers are used.

DATA MODELS, SCHEMAS, AND INSTANCES (2)

- A **data model**- is a collection of concepts that can be used to describe the structure (data types, relationships, and constraints) of a database.
- Data models include a set of basic operations for specifying **retrievals and updates** on the database.
- **User-defined** operation - COMPUTE_GPA
- **Basic data model operations** - insert, delete, modify, or retrieve

DATA MODELS, SCHEMAS, AND INSTANCES (3)

Categories of Data Models (1)

- **High-level or conceptual data models** - close to the way many users perceive data, for **end users**.
- **low-level or physical data models** - describe the details of **how data is stored in the computer** - meant for computer specialists.
- **Representational or implementation data models** - **hide some details of data storage** but can be implemented on a computer system in a direct way. .(used in traditional commercial DBMSs)
- **Conceptual data models** - use concepts such as **entities, attributes, and relationships**. **Entity** represents a **real-world object or concept**, such as an employee or a project, that is described in the database.
- An **attribute** represents some property of interest that further **describes an entity**, such as the employee's name or salary.
- A **relationship** among two or more entities represents an **association** among two or more entities

DATA MODELS, SCHEMAS, AND INSTANCES (4)

Categories of Data Models (2)

- **Physical data models** describe how **data is stored as files in the computer** by representing information such as **record formats, record orderings, and access paths**.
- An **access path** is a structure that **makes the search for particular database records efficient**.
- Representational or implementation data models include **relational data model**, the **network** and **hierarchical** models. They represent data by using record structures and hence are sometimes called record-based data models.
- **Object data models** are also frequently utilized as **high-level conceptual models**, particularly in the software engineering domain.

DATA MODELS, SCHEMAS, AND INSTANCES (5)

Schemas, Instances, and Database State(1)

- **Database schema** - The **description of a database** which is specified during database design and is not expected to change frequently.
- **Schema diagram** - A **displayed schema** is called a schema diagram.
- **Schema construct** - each **object** in the schema ex:- STUDENT or COURSE.
- **Database state or snapshot** - The data in the **database at a particular moment in time**. It is also called the *current* set of occurrences or instances in the database
- The schema is sometimes called the **intension**, and a database state an **extension** of the schema.
- **Schema evolution** – **changes** made to schema, for ex:- adding new attribute DoB.

DATA MODELS, SCHEMAS, AND INSTANCES (6)

Schemas, Instances, and Database State(2)

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

Figure 2.1

Schema diagram for the database in Figure 1.2.

THREE-SCHEMA ARCHITECTURE AND DATA INDEPENDENCE (1)

The Three Schema Architecture (1)

- Proposed to achieve and visualize the characteristics
- The goal of the three-schema architecture, is to **separate the user applications and the physical database.**
- In this architecture, schemas can be defined at the following **three levels:**

1. The internal level

- has an **internal schema**, which describes the **physical storage structure** of the database.
- uses a **physical data model** and
- describes the complete details of **data storage and access paths** for the database.

THREE-SCHEMA ARCHITECTURE AND DATA INDEPENDENCE (2)

The Three Schema Architecture (2)

- **2. The conceptual level**
 - has a **conceptual schema**, which **describes the structure of the whole database** for a community of users.
 - **hides the details of physical storage** structures and concentrates on **describing entities, data types, relationships, user operations, and constraints**.
 - **representational data model** is used to describe the conceptual schema
 - This *implementation conceptual schema* is often based on a *conceptual schema design* in a high-level data model.

THREE-SCHEMA ARCHITECTURE AND DATA INDEPENDENCE (3)

The Three Schema Architecture (3)

3. The external or view level

- includes a number of **external schemas or user views**.
- describes the part of the database that a **particular user group is interested in** and hides the rest of the database from that user group.
- implemented using a **representational data model**, possibly based on an **external schema design** in a highlevel data model.

Most DBMSs **do not separate** the three levels completely.

THREE-SCHEMA ARCHITECTURE AND DATA INDEPENDENCE (4)

The Three Schema Architecture (4)

- In a DBMS based on the three-schema architecture, each user group refers only to its own external schema. Hence, the DBMS must **transform a request specified on an external schema into a request against the conceptual schema**, and then into a **request on the internal schema** for processing over the stored database.
- If the request is a database retrieval, the data extracted from the stored database must be **reformatted to match the user's external view**.
- The **processes of transforming requests and results** between levels are called **mappings**. Mapping is not good for small DBMS because it takes more time. In External / Conceptual mapping, it is necessary to transform the request from external level to conceptual schema.



THREE-SCHEMA ARCHITECTURE AND DATA INDEPENDENCE (5)

The Three Schema Architecture (5)

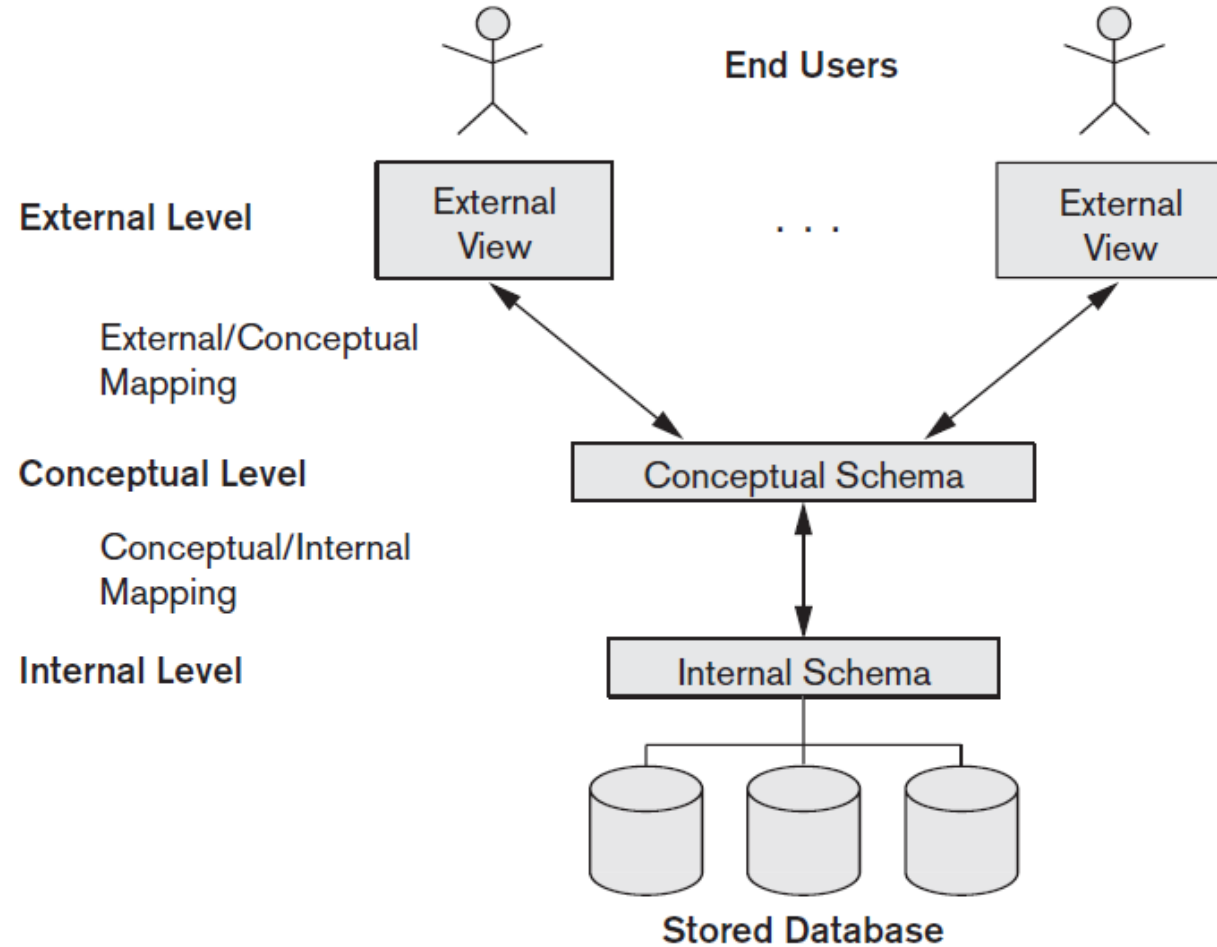


Figure 2.2

The three-schema architecture.

THREE-SCHEMA ARCHITECTURE AND DATA INDEPENDENCE (6)

Data Independence (1)

- Data independence can be defined as the **capacity to change the schema at one level** of a database system **without having to change the schema at the next higher level**.

1. Logical data independence

- is the capacity to **change the conceptual schema without having to change external schemas** or application programs.
- We may change the conceptual schema to **expand the database** (by adding a record type or data item), to **change constraints**, or to **reduce the database** (by removing a record type or data item).
- Only the **view definition and the mappings** need be **changed** in a DBMS that supports logical data independence

THREE-SCHEMA ARCHITECTURE AND DATA INDEPENDENCE (7)

Data Independence (2)

2. Physical data independence

- is the **capacity to change the internal schema without having to change the conceptual schema.**
- Changes to the internal schema may be needed because some **physical files had to be reorganized-** for example, by creating additional access structures-to improve the performance of retrieval or update.
- If the same data as before remains in the database, we should not have to change the conceptual schema. ex:- change of access structure.
- Data independence occurs because when the schema is changed at some level, the schema at the next higher level remains unchanged; **only the *mapping* between the two levels is changed.**

DATABASE LANGUAGES AND INTERFACES (1)

DBMS LANGUAGES (1)

- **Data definition language (DDL)**, is used by the DBA and by database designers to define both conceptual and internal schemas when there is no strict separation of the levels.
- The DBMS will have a DDL compiler which process DDL statements in order to identify descriptions of the schema constructs and to store the schema description in the DBMS catalog.
- In DBMSs where a clear separation is maintained between the conceptual and internal levels, the DDL is used to specify the conceptual schema only.
- **Storage Definition Language (SDL)**, is used to specify the internal schema.
- The mappings between the two schemas may be specified in either DDL or SDL languages.
- **View Definition Language (VDL)**, is used to specify user views and their mappings to the conceptual schema, but in most DBMSs the DDL is used to define both conceptual and external schemas.

Data Definition Language (DDL)

Specification notation for defining the database schema

Example: **create table** *instructor* (
 ID **char**(5),
 name **varchar**(20),
 dept_name **varchar**(20),
 salary **numeric**(8,2))

DDL compiler generates a set of table templates stored in a ***data dictionary***

Data dictionary contains metadata (i.e., data about data)

- Database schema
- Integrity constraints
 - Primary key (ID uniquely identifies instructors)
 - Referential integrity (**references** constraint in SQL)
 - e.g. *dept_name* value in any *instructor* tuple must appear in *department* relation
- Authorization

DATABASE LANGUAGES AND INTERFACES (2)

DBMS LANGUAGES (2)

- **Data Manipulation Language (DML)** is used to manipulated database which includes retrieval, insertion, deletion, and modification of the data.
- A **comprehensive integrated language** is used that includes constructs for conceptual schema definition, view definition, ami data manipulation.
- **SQL relational database language** is a comprehensive database language which represents a combination of DDL, VDL, and DML.
- There are two main types of DMLs.
- A **high-level or nonprocedural DML** can be used on its own to specify complex database operations in a concise manner.
- Many DBMSs allow high-level DML statements either to be entered **interactively** from a display monitor or terminal or to be **embedded** in a general-purpose programming language.

DATABASE LANGUAGES AND INTERFACES (3)

DBMS LANGUAGES (3)

- In the latter case, DML statements must be identified **within the program** so that they can be extracted by a **precompiler** and processed by the DBMS.
- A **low-level or procedural DML must be embedded** in a general-purpose programming language. This type of DML typically retrieves **individual records** or objects from the database and processes each separately.
- Hence, it needs to use programming language constructs, such as **looping**, to retrieve and process each record from a set of records.
- Low-level DMLs are also called **record-at-a-time DMLs** because of this property.
- High-level DMLs, such as **SQL**, can specify and retrieve many records in a single DML statement and are hence called **set-at-a-time or set-oriented DMLs**.

DATABASE LANGUAGES AND INTERFACES (4)

DBMS LANGUAGES (4)

- A query in a high-level DML often specifies **which data to retrieve rather than how to retrieve it**; hence, such languages are also called **declarative**.
- Whenever DML commands, whether high level or low level, are **embedded** in a general-purpose programming language, that language is called the **host language** and the DML is called the data **sublanguage**.
- On the other hand, a **high-level DML** used in a **stand-alone interactive** manner is called a query language.
- **Casual end users** typically use a **high-level query language** to specify their requests, whereas **programmers** use the **DML in its embedded form**.

DATABASE LANGUAGES AND INTERFACES (5)

DBMS Interface (1)

- **Menu-Based Interfaces for Web Clients or Browsing** – present the user with lists of options, called **menus**, that lead the user through the formulation of a request. **No need to memorize the specific commands** and syntax of a query language; rather, the query is composed step by step by picking options from a menu that is displayed by the system. **Pull-down menus** are a very popular technique in **Web-based user interfaces**. Eg: ATM
- **Forms-Based Interfaces** - displays a **form** to each user. **Users can fill out** all of the form entries to **insert new data**, or they fill out only certain entries, in which case the DBMS will retrieve matching data for the remaining entries. Forms are usually designed and programmed for **naive users** as interfaces to canned transactions. Many DBMSs have **forms specification languages**, which are special languages that help programmers specify such forms.

DATABASE LANGUAGES AND INTERFACES (6)

DBMS Interface (2)

- **Graphical User Interfaces** - displays a schema to the user in **diagrammatic form**. The user can then specify a query by manipulating the diagram. In many cases, GUIs utilize both **menus and forms**. Most GUIs use a **pointing device**, such as a **mouse**, to pick certain parts of the displayed schema diagram.
- **Natural Language Interfaces** - accept requests written in **English** or some **other language** and attempt to "understand" them. A natural language interface usually has its **own "schema,"** which is similar to the database conceptual schema, as well as a **dictionary of important words**. The natural language interface refers to the words in its schema, as well as to the set of standard words in its dictionary, to **interpret the request**. If the interpretation is successful, the interface generates a **high-level query** corresponding to the natural language request and submits it to the DBMS for processing; otherwise, **a dialogue is started with the user to clarify the request**. Eg: Siri, Alexa, Google Assistant or Cortana

DATABASE LANGUAGES AND INTERFACES (7)

DBMS Interface (3)

- **Interfaces for Parametric Users** - a small set of abbreviated commands is included, with the goal of **minimizing the number of keystrokes** required for each request. For example, **function keys** in a terminal can be programmed to initiate the various commands. This allows the parametric user to proceed with a minimal number of keystrokes.
- **Interfaces for the DBA.** Most database systems contain **privileged commands** that can be used only by the DBA's staff. These include commands for **creating accounts, setting system parameters, granting account authorization, changing a schema, and reorganizing the storage structures of a database.**

THE DATABASE ENVIRONMENT (1)

DBMS Component Modules (1)

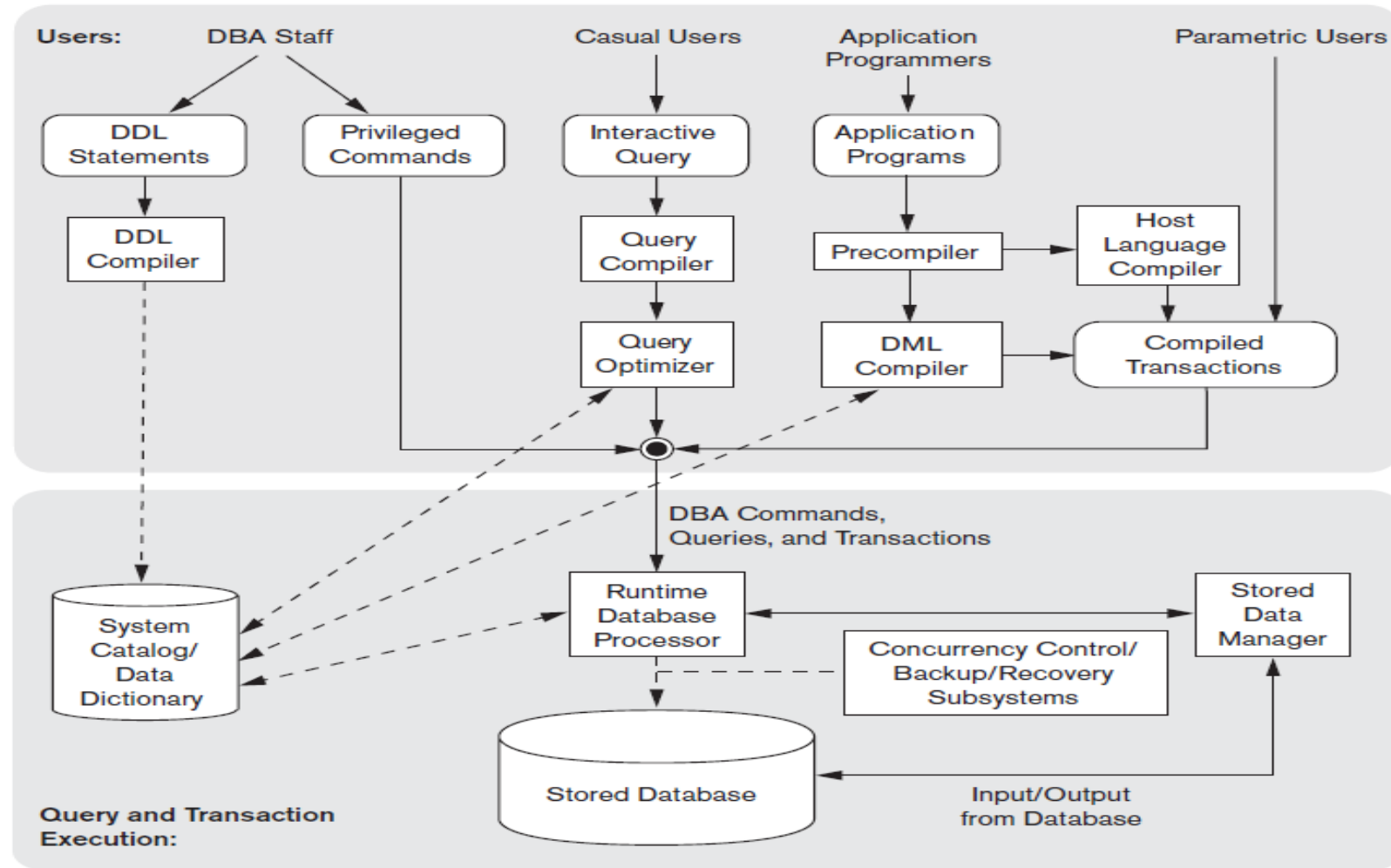


Figure 2.3
Component modules of a DBMS and their interactions.

THE DATABASE ENVIRONMENT (2)

DBMS Component Modules (2)

- Figure 2.3 illustrates, in a simplified form, the typical **DBMS components**.
- The **database** and the DBMS **catalog** are usually **stored on disk**.
- A higher-level **stored data manager module** of the DBMS controls **access to DBMS information that is stored on disk**, whether it is part of the database or the catalog.
- The stored data manager may **use basic os services for carrying out lowlevel data transfer** between the disk and computer main storage, but it **controls** other aspects of data transfer, such as **handling buffers in main memory**.
- The **DDL compiler processes schema definitions**, specified in the DDL, and **stores** descriptions of the schemas (**meta-data**) in the DBMS **catalog**. The catalog includes information such as the names and sizes of files, names and data types of data items, storage details of each file, mapping information among schemas, and constraints, in addition to many other types of information that are needed by the DBMS modules.

THE DATABASE ENVIRONMENT (3)

DBMS Component Modules (3)

- The **runtime database processor** handles **database accesses at runtime**; it receives retrieval or update operations and carries them out on the database.
- The **query compiler** handles **high-level queries** that are entered **interactively**. It **parses, analyzes, and compiles or interprets a query** by creating database access code, and then **generates calls to the runtime processor** for executing the code.
- The **precompiler** extracts **DML commands from an application program** written in a host programming language. These commands are sent to the **DML compiler** for compilation into object code for database access. The **rest** of the program is sent to the **host language compiler**. The **object codes** for the **DML commands** and the **rest of the program are linked**, forming a canned transaction whose executable code includes calls to the runtime database processor.

THE DATABASE ENVIRONMENT (4)

DBMS Component Modules (4)

- The **DBMS interacts with the operating system** when **disk accesses**-to the **database** or to the **catalog**-are needed.
- If the computer system is **shared by many users**, the **os will schedule** DBMS disk access requests and DBMS processing along with other processes.
- On the other hand, if the computer system is mainly **dedicated to running the database server**, the **DBMS will control main memory buffering of disk pages**.
- The **DBMS also interfaces** with **compilers** for general-purpose host programming languages, and with **application servers** and **client programs** running on separate machines through the **system network interface**.
-

THE DATABASE ENVIRONMENT (5)

Database System Utilities (1)

- DBMSs have database utilities that help the **DBA** in managing the database system. Common utilities have the following types of functions:
- **Loading:** A loading utility is used to **load existing data files-such as text files or sequential files-into the database**. Usually, the current **(source) format** of the data and the **desired (target) database** file structure are **specified** to the **utility**, which then **automatically reformats** the data and stores it in the database. Some **vendors** are offering products that generate the appropriate loading programs, given the existing source and target database storage descriptions (internal schemas). Such tools are also called **conversion tools**.
- **Backup:** A backup utility creates a **backup copy of the database**, usually by dumping the entire database onto **tape**. The backup copy can be used to restore the database in case of **catastrophic failure**. **Incremental backups** are also often used, where only changes since the previous backup are recorded. Incremental backup is **more complex but saves space**.

THE DATABASE ENVIRONMENT (6)

Database System Utilities (2)

- ***File reorganization:*** This utility can be used to **reorganize a database file** into a different file organization to **improve performance**.
- ***Performance monitoring:*** **monitors database usage** and provides **statistics to the DBA**. The DBA uses the statistics in making decisions such as whether or not to reorganize files to improve performance.

Other utilities may be available for **sorting files, handling data compression, monitoring access by users, interfacing with the network**, and performing other functions.

THE DATABASE ENVIRONMENT (7)

Tools, Application Environments, and Communications Facilities (1)

- Other tools are often available to database designers, users, and DBAs.
- CASE tools are used in the design phase of database systems.
- Expanded **data dictionary (or data repository)** system. In addition to storing catalog information about schemas and constraints, the data dictionary stores other information, such as **design decisions, usage standards, application program descriptions, and user information**. Such a system is also called an **information repository**. This information can be accessed *directly* by users or the DBA when needed.
- A data dictionary utility is similar to the DBMS catalog, but it includes a **wider variety of information** and is **accessed mainly by users** rather than by the DBMS software.

THE DATABASE ENVIRONMENT (8)

Tools, Application Environments, and Communications Facilities (2)

- **Application development environments**, such as the **PowerBuilder (Sybase)** or **JBuilder (Borland)** system, are becoming quite popular. These systems provide an environment for **developing database applications** and include facilities that help in many facets of database systems, including **database design, GUI development, querying and updating, and application program development**.
- The DBMS also needs to interface with **communications software**, whose function is to **allow users at locations remote from the database system site to access the database** through computer terminals, workstations, or their local personal computers. These are **connected** to the database site through data communications hardware such as **phone lines, long-haul networks, local area networks, or satellite communication devices**.
- Many commercial database systems have **communication packages** that work with the DBMS. **The integrated DBMS and data communications system is called a DB/DC system**. In addition, some **distributed DBMSs** are **physically distributed over multiple machines**. **LANs** are used to connect them.

END OF CHAPTER 2